Promethee Web Application

A Project Report submitted for the degree of

Computer Science

Prepared by

Antonin Marouzé

Under the supervision of

Georgios Bardis

Department of Computer Science

University Of West Attika

2026

Contents

List of Figures

CHAPTER 1

Introduction

1.1 Internship Context

I am currently completing my second year of a Bachelor Universitaire de Technologie (BUT) in Computer Science at the Institut Universitaire de Technologie (IUT) of Lille, France. As a mandatory part of this degree, representing 15 ECTS units, students are required to complete a professional internship. This mobility period within the Erasmus+ program allowed me to undertake this international experience at the University of West Attica.

1.2 Mobility and Motivation

I chose this Erasmus+ mobility to immerse myself in a new academic and professional culture, broaden my international perspective, and improve my technical English proficiency within a research and development environment.

1.3 Multi-Criteria Decision Analysis (MCDA)

Multi-Criteria Decision Analysis (MCDA) encompasses a collection of methodologies designed to evaluate and select alternatives based on multiple, often conflicting, criteria. These methods are widely employed by organisms and companies for decision support. MCDA methods aim to evaluate alternatives either in an absolute manner (providing a complete ranking) or in a relative manner (identifying strict preference, indifference, or incomparability between specific pairs). Furthermore, some methods allow for grouping alternatives into similarity groups based on shared characteristics.

1.4 Project Objectives

The primary objective of this internship, as defined in the project specification provided by the supervisor, was to design and develop a complete web application implementing both PROMETHEE I and PROMETHEE II methodologies, allowing a Decision Maker to input a performance matrix, define criteria with specific preference functions and weights, and obtain both a partial ranking (PROMETHEE I) and a complete ranking (PROMETHEE II) of the alternatives. The final deliverables include the full application source code, comprehensive documentation, and a working demonstration of the platform.

1.5 Report Organization

The first chapter of the report contains the introduction and the context of the internship. The second chapter presents the MCDA methods and the theoretical framework of the

PROMETHEE algorithm. The third chapter outlines the functional design and use cases of the application. The fourth chapter discusses the technical architecture and implementation choices. The fifth chapter presents an extended usage example and a discussion on the application's strengths. Finally, the sixth chapter concludes the report with a synopsis of the work and future perspectives.

CHAPTER 2

MCDA Methods and Project Subject

2.1 Basics of MCDA Methods

In the realm of MCDA, a Decision Maker (DM) expresses their preferences to evaluate a finite set of alternatives. These alternatives are judged across various attributes organized as criteria (e.g., price, area, safety rating, condition). A Decision Analyst (DA) typically guides the DM through this process, organizing information and applying mathematical models.

MCDA is broadly divided into two main schools of thought:

- **Utility Function Methods (American School):** Represented by weighted sum methodologies and the Analytic Hierarchy Process (AHP).
- **Outranking Methods (European School):** Focused on pairwise comparisons, with main representatives being ELECTRE and PROMETHEE.

These two families cover the vast majority of MCDA applications encountered in practice [2].

2.2 Implemented Method: PROMETHEE Algorithm

The PROMETHEE method (Preference Ranking Organization Method for Enrichment of Evaluations) was introduced by Brans and Vincke [1] and is based on the principle of pairwise comparisons.

2.2.1 Unicriterion Preference

For a criterion j, we calculate the difference between the evaluation of alternative a and alternative b:

$$d_j(a,b) = g_j(a) - g_j(b) \tag{2.1}$$

This difference is transformed into a preference degree $P_j(a,b) \in [0,1]$ using one of the six standard preference functions:

1. **Type 1 (Usual):** $P(d) = 1$ if $d > 0$, else 0.
2. **Type 2 (U-Shape):** $P(d) = 1$ if $d > q$, else 0 (uses indifference threshold q).
3. **Type 3 (V-Shape):** $P(d) = d/p$ if $0 < d \leq p$, and 1 if $d > p$ (uses preference threshold p).
4. **Type 4 (Level):** $P(d) = 0.5$ if $q < d \leq p$, and 1 if $d > p$.

5. **Type 5 (V-Shape Indiff.):** $P(d) = (d-q)/(p-q)$ if $q < d \leq p$, and 1 if $d > p$.
6. **Type 6 (Gaussian):** $P(d) = 1 - e^{-d^2/2s^2}$ (uses standard deviation s).

2.2.2 Global Preference and Flows

The aggregated preference index $\pi(a, b)$ is calculated using criterion weights w_j:

$$\pi(a,b) = \sum_{j=1}^{k} w_j P_j(a,b) \tag{2.2}$$

where the weights w_j are normalized such that $\sum_{j=1}^{k} w_j = 1$ and $w_j \geq 0$.

PROMETHEE I calculates the positive flow $\Phi^+(a)$ (dominance power) and the negative flow $\Phi^-(a)$ (weakness):

$$\Phi^+(a) = \frac{1}{n-1} \sum_{x \neq a} \pi(a,x) \quad , \quad \Phi^-(a) = \frac{1}{n-1} \sum_{x \neq a} \pi(x,a) \tag{2.3}$$

2.2.3 PROMETHEE I vs PROMETHEE II

PROMETHEE I establishes a partial ranking. Alternative a is preferred to b (aP^Ib) if $\Phi^+(a) \geq \Phi^+(b)$ and $\Phi^-(a) \leq \Phi^-(b)$, with at least one strict inequality. If $\Phi^+(a) > \Phi^+(b)$ and $\Phi^-(a) > \Phi^-(b)$ (or vice-versa), the alternatives are considered **incomparable**. Two alternatives a and b are considered **indifferent** (aI^Ib) if $\Phi^+(a) = \Phi^+(b)$ and $\Phi^-(a) = \Phi^-(b)$.

PROMETHEE II establishes a complete ranking by calculating the **Net Flow**:

$$\Phi(a) = \Phi^+(a) - \Phi^-(a) \tag{2.4}$$

The alternative with the highest net flow is ranked first.

Functional Design

3.1 User Interface and Features

The application provides a dynamic web interface allowing users to define decision problems without technical expertise. The layout is divided into a data entry matrix, a pairwise comparison tool, and a final results table.

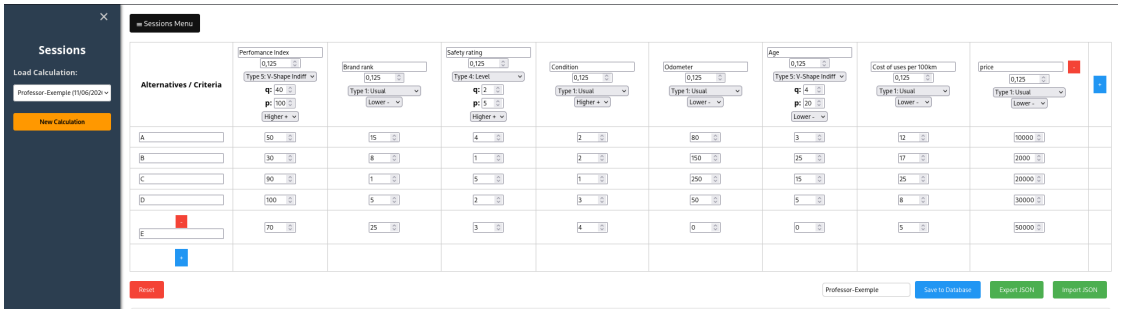

Figure 3.1: Data Entry Interface: Users can dynamically add criteria and alternatives, choosing weights and preference functions through simple dropdowns and inputs.

Key functionalities include:

- **Dynamic Matrix:** Real-time addition/removal of criteria and alternatives.
- **Real-time Calculation:** The results update automatically as soon as the criterion weights sum to 1.0.
- **Pairwise Analysis:** A dedicated section to compare any two alternatives criterion-by-criterion.

The interface is designed to support both roles identified in MCDA methodology: the **Decision Maker (DM)**, who inputs evaluations and interprets results, and the **Decision Analyst (DA)**, who configures the criteria, weights, and preference functions.

3.2 Usage Scenario and Use Cases

Consider a Decision Maker wishing to select a company vehicle. They open the application and create a new session named “Vehicle Purchase 2026”. They add three alternatives (Car A, Car B, Car C) and define three criteria: Price (to minimize), Quality (to maximize), and Delivery Time (to minimize). For each criterion, they select an appropriate preference function and assign weights summing to 1.0. As they fill in the evaluation scores, the PROMETHEE II ranking updates in real time. Finally, they use the pairwise comparison tool to analyze why Car A ranks above Car C on specific criteria, and save the session for future reference.

The full set of user interactions and functionalities described in this scenario are formalized in the Use Case diagram below (see Figure 3.2).

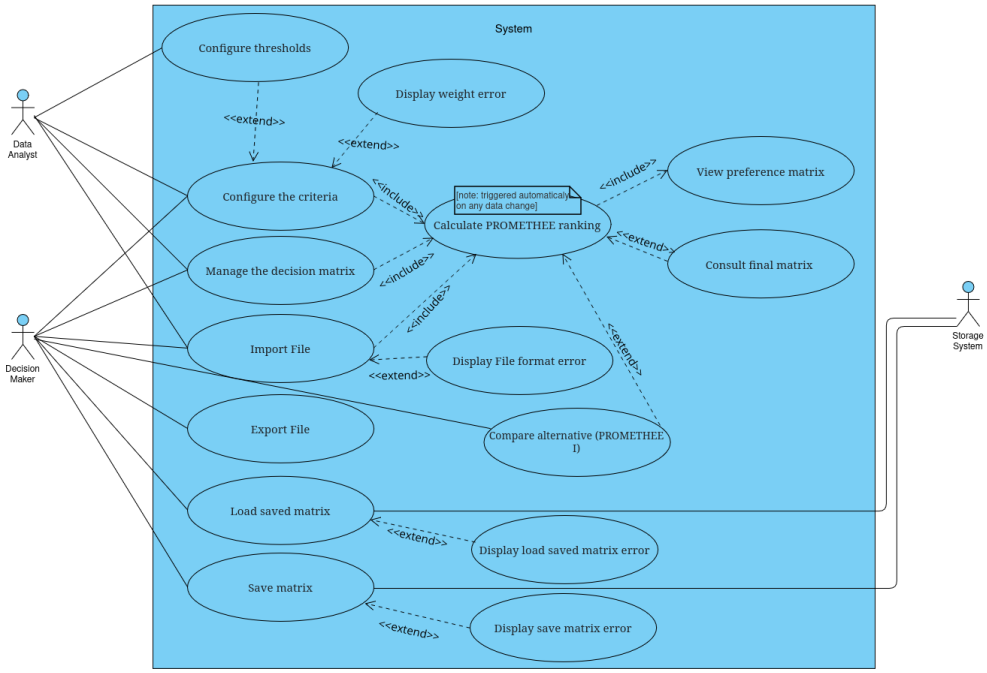

Figure 3.2: Use Case Diagram: Illustrates the primary interactions a user has with the system, such as managing the matrix and exporting results.

3.3 Data Storage Overview

The application persists four types of information to ensure full persistence: **Sessions** (the global decision problem), **Criteria** (weights and preference function parameters), **Alternatives** (names and identities), and **Evaluations** (the raw performance scores linking each alternative to each criterion). The detailed relational schema is presented in the Technical Design chapter.

CHAPTER 4

Technical Design

4.1 Implementation Choices and Technologies

The backend is implemented using **Jakarta EE Servlets**, chosen for its robustness and my strong academic familiarity with the Java ecosystem. **Apache Tomcat 10.1** was selected for its open-source nature and widespread industry adoption as a stable Servlet container. For data persistence, **PostgreSQL** was preferred over alternatives for its strict ACID compliance and advanced support for relational data. Database connectivity is ensured via the **PostgreSQL JDBC Driver**, which enables the Jakarta EE application to communicate with the database using standard SQL queries. Finally, **Docker** was employed to ensure a fully reproducible deployment environment across different machines.

The frontend utilizes vanilla JavaScript and JavaServer Pages (JSP) to maintain high performance and avoid the overhead of heavy frameworks. To improve the user experience, the application utilizes the browser's **localStorage** to automatically cache the matrix state during editing.

4.2 System Architecture and Deployment

The application follows a classic three-tier architecture, physically distributed across multiple nodes. As detailed in the Deployment Diagram (see Figure 4.1), the system consists of a client browser, an application server container, and a database container.

The two API endpoints are strictly separated by responsibility:

- `PrometheeServlet` (`/calculate`) is stateless and processes raw JSON matrix data to return computed flows.
- `SessionServlet` (`/api/sessions`) manages stateful CRUD operations against the PostgreSQL database.

The entire stack is orchestrated using Docker Compose, which seamlessly networks the Tomcat application container with the database container, mounting a local volume for persistent storage and initializing the schema automatically.

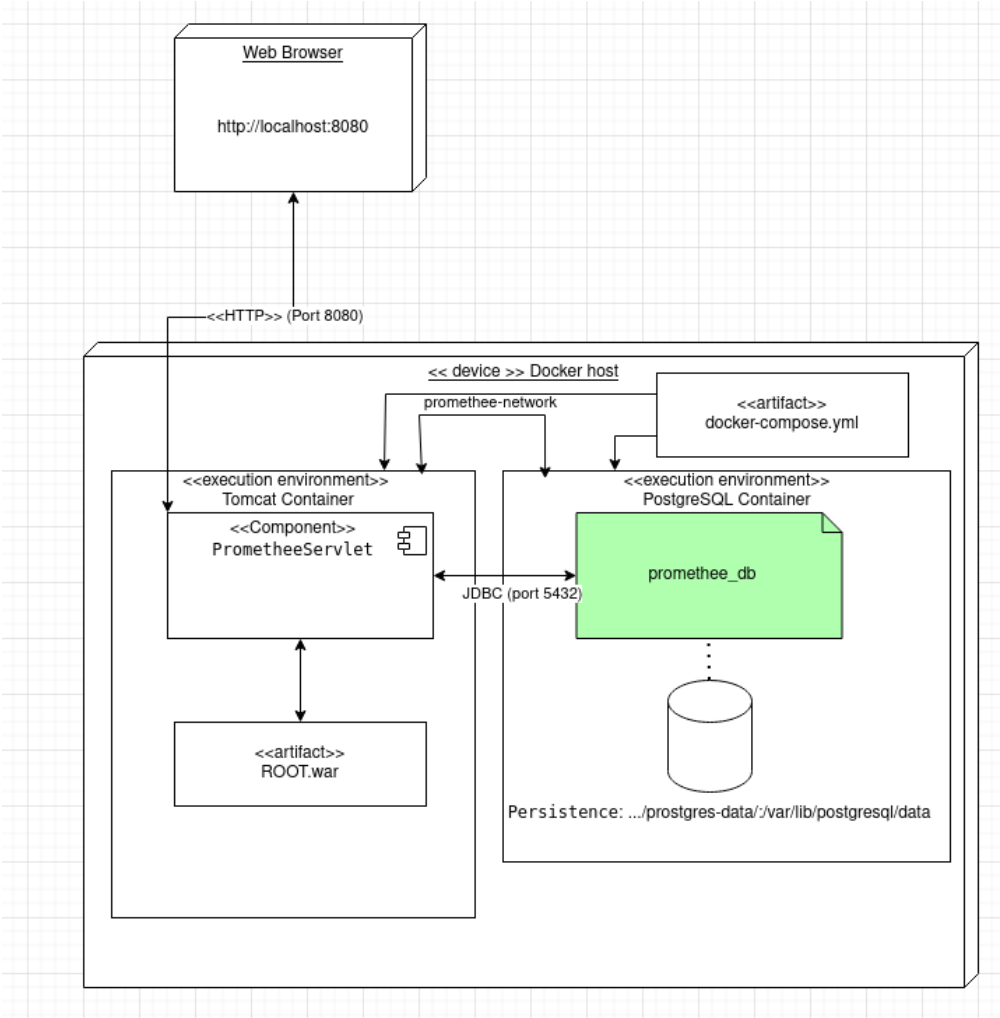

Figure 4.1: Deployment Diagram: Shows the physical distribution of the system components.

4.3 Logical Data Model (MLD)

The relational structure of the database was designed to support the PROMETHEE data model, ensuring a clear separation between session metadata, criteria parameters, and raw evaluation scores.

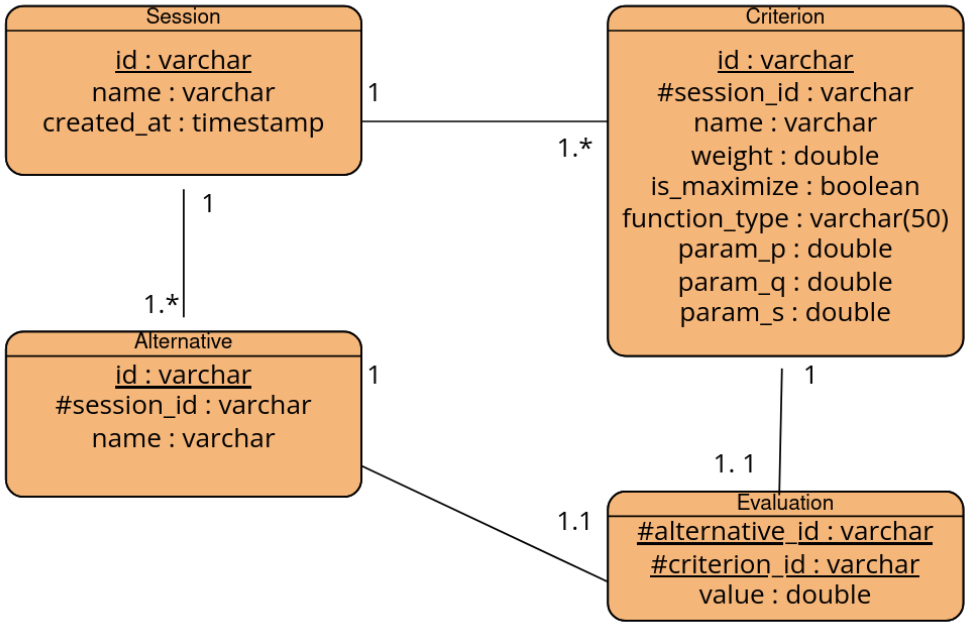

Figure 4.2: Logical Data Model: Relational schema including primary and foreign key associations.

4.4 MVC and Service Layer Architecture

To ensure maintainability and scalability, the code follows a strict Model-View-Controller (MVC) pattern, enhanced with a dedicated Service layer to decouple concerns.

- **Controller:** Managed by Jakarta EE Servlets (`PrometheeServlet` and `SessionServlet`), which handle incoming HTTP requests and delegate logic to the service layer.
- **Service:** The `PrometheeService` centralizes the business logic, including JSON data parsing (utilizing the powerful **Jackson** library) and the orchestration of the mathematical calculation engine. It also implements the **DTO (Data Transfer Object)** pattern via the `CalculationResult` class to efficiently package API responses.
- **Model:** Classes (`Alternative`, `Criterion`, and `Session`) encapsulate the PROMETHEE data structures and mathematical rules.
- **DAO:** The `SessionDAO` handles interaction with the PostgreSQL database using standard JDBC.

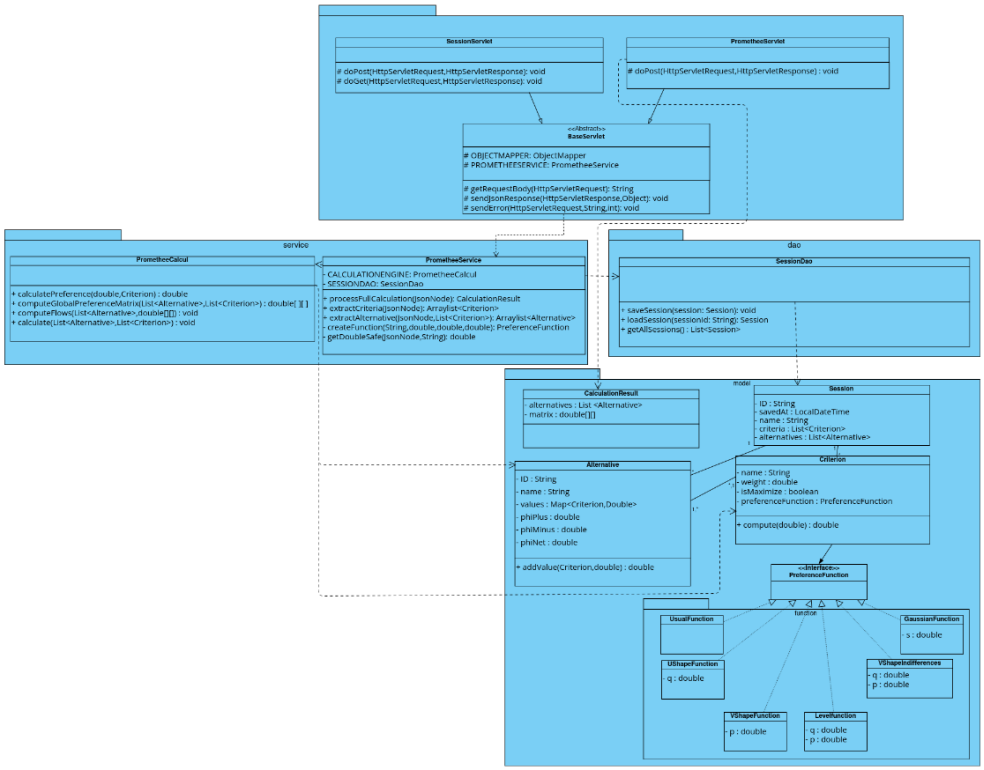

Figure 4.3: Class Diagram: Internal object-oriented structure of the mathematical model.

CHAPTER 5

Discussion

5.1 Extended Experiment and Usage Example

To validate the application, a vehicle selection scenario was implemented. Three alternatives were compared against three criteria using the parameters defined in Table 5.1.

Table 5.1: Experiment Data: Input Matrix and Parameters

Alternative Weight	Price (Min, V-Shape) w=0.5, p=10	Quality (Max, Usual) w=0.3	Delay (Min, U-Shape) w=0.2, q=5
Car A	80	10	15
Car B	90	9	12
Car C	75	8	20

The application generated the following results (see Figure 5.1):

Table 5.2: Experiment Results: Calculated Flows

Alternative	Φ^+ (Power)	Φ^- (Weakness)	Φ (Net Flow)	Rank
Car A	0.5500	0.1250	0.4250	1
Car C	0.3750	0.4000	-0.0250	2
Car B	0.2500	0.6500	-0.4000	3

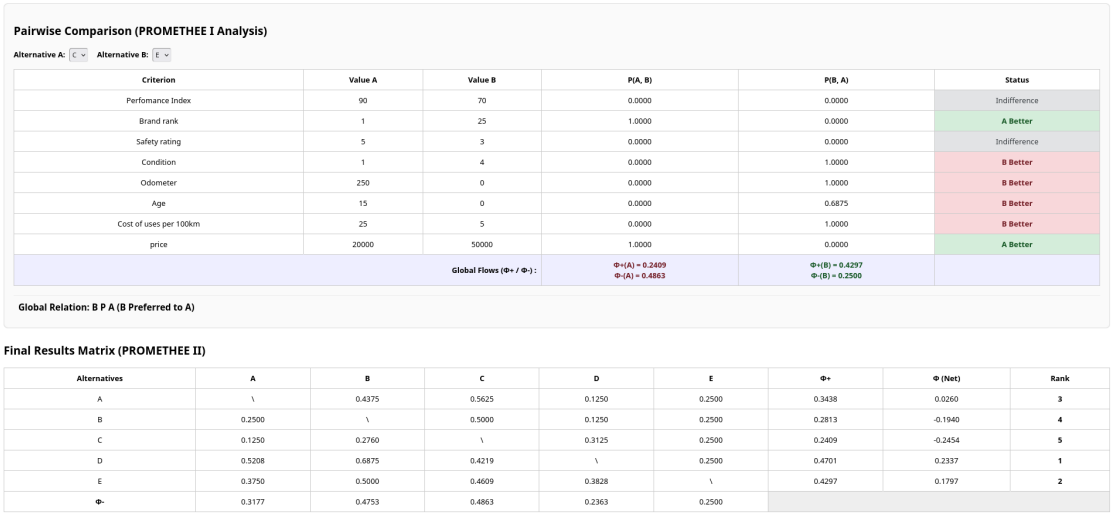

Figure 5.1: Results Interface: Final ranking and pairwise comparison between Car A and Car C.

The results confirm that Car A is the best overall choice, driven primarily by its competitive price (dominant criterion with w=0.5). Car C ranks second despite its

lower price because its significantly longer delivery time (20 days vs 15 for Car A) penalizes it on that criterion. Car B ranks last due to its highest price (90), which is decisively penalized by the V-Shape function with p=10.

Applying PROMETHEE I, Car A and Car C are comparable ($\Phi^+(A) > \Phi^+(C)$ and $\Phi^-(A) < \Phi^-(C)$), confirming Car A's dominance. However, Car A and Car B are also comparable, while Car B and Car C present an interesting case worth examining in the pairwise tool.

5.2 Analysis: Positives and Limitations

Positives: The application provides a highly responsive experience through real-time AJAX recalculations. The backend follows SOLID principles and utilizes the DTO pattern for API communication. The application also supports JSON import/export, enabling users to share and reload decision matrices across sessions.

Limitations and Future Alternatives: One observed limitation is that the current frontend architecture does not provide visual feedback when weights do not sum to exactly 1.0 due to floating-point precision issues (e.g., 0.1 + 0.2 + 0.7 = 0.9999... in JavaScript). A rounding mechanism was implemented as a workaround, but a more robust solution would involve server-side weight normalization.

Graphical visualizations such as the GAIA plane [3] would allow Decision Makers to better interpret conflicting criteria visually. Furthermore, adding multi-user authentication would transform the tool into a collaborative platform supporting multiple Decision Makers simultaneously.

CHAPTER 6

Conclusions

6.1 Synopsis of Work

This project delivered a fully functional, containerized web application implementing the PROMETHEE I and II multi-criteria decision analysis methods. The platform provides a dynamic interface for defining complex decision problems, evaluating alternatives through six distinct preference functions, and persistently storing results in a relational PostgreSQL database. The entire stack is containerized with Docker, ensuring reproducible deployment. The backend follows SOLID principles through a strict MVC architecture with a dedicated Service layer and DAO pattern.

6.2 Suggestions for Improvement

Future iterations should focus on four axes: (1) modernizing the frontend architecture for better state management; (2) adding the GAIA plane visualization [3] for richer result interpretation; (3) implementing user authentication for multi-user support; and (4) adding server-side weight normalization to eliminate floating-point precision issues.

6.3 Personal Impressions

Working with Jakarta EE Servlets deepened my understanding of the HTTP request-response lifecycle and stateless API design. Implementing the six preference functions required carefully translating mathematical specifications into robust, tested Java code — a level of rigor I had not previously exercised. Designing the Docker Compose orchestration also introduced me to professional deployment practices. The Erasmus+ context further challenged me to communicate all technical concepts in English daily, which significantly improved my professional fluency and adaptability in an international environment.

Bibliography

[1] J.P. Brans and P. Vincke, "A Preference Ranking Organisation Method: (The PROMETHEE Method for Multiple Criteria Decision-Making)," *Management Science*, vol. 31, no. 6, pp. 647–656, 1985.

[2] V. Belton and T. Stewart, *Multiple Criteria Decision Analysis: An Integrated Approach*. Boston: Kluwer Academic Publishers, 2002.

[3] B. Mareschal, "The PROMETHEE Methods," in *Multiple Criteria Decision Analysis: State of the Art Surveys*, J. Figueira, S. Greco, and M. Ehrgott, Eds. New York: Springer, 2013.

[4] Eclipse Foundation, "Jakarta EE 10 Documentation," [Online]. Available: https://jakarta.ee/specifications/platform/10/.

[5] FasterXML, "Jackson Data Processor Documentation," [Online]. Available: https://github.com/FasterXML/jackson-docs.

[6] Apache Software Foundation, "Apache Tomcat 10.1 Documentation," [Online]. Available: https://tomcat.apache.org/tomcat-10.1-doc/.

[7] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: https://www.postgresql.org/docs/16/index.html.

[8] Docker Inc., "Docker Documentation," [Online]. Available: https://docs.docker.com/.